

1 Background

In Lab 7, we wrote a multiplication program in RISC-V, which we translated to compressed machine code to run on our computer. We also made changes to the program counter module to support branching. This lab represented the full functionality of our system.

2 Prelab

The prelab had us write a RISC-V program that could multiply two signed 8-bit numbers. Our program can be found in Appendix A.1. Given the directions, the initial program we designed loaded 2 values from memory, using 1 as the number of iterations in a loop. The other value was added to itself within the loop. We had several branch conditions to determine the signs of both numbers using `andi`, `xor`, and `addi`. This program was tested in the RARS RISC-V simulator, where it successfully multiplied negative and positive numbers in both positions.

3 Design

3.1 Program Counter

In Lab 7, we extended the width of the pc output to 9 bits, aligned with the 9-bit offset input, so branch instructions could be implemented. We added a condition for `take_branch`, where if `take_branch` was enabled the program counter would increment by offset.

3.2 Instruction Set

We used a program on the COE virtual Linux machines to translate the RISC-V instructions to machine code. Unfortunately, we encountered issues when translating the program we designed. All `andi` instructions did not translate to machine code, and the same occurred with `lw` instructions. The words we stored in memory did not carry over to the SystemVerilog implementation of the RISC-V computer. In the end, we rewrote the multiply instruction set to incorporate fewer loops and use no `lw` instructions. The modified machine code can be found in Appendix A.2.

4 Evaluation

4.1 Physical Design

The system connections were created in the top-level file, and the VIO and data memory were instantiated before testing the design. In Lab 7, the VIO was only used to monitor the status of different system wires. On the board, pushbutton 1 was used to control the system clock and iterate through the instruction list, and pushbutton 0 was used as a reset button for the registers and program counter. The VIO and board outputs for instructions 3-7 can be found in Figures 1, 2, 3, 4, and 5 respectively. The machine code can be found in Appendix A.3. Our design's functionality was verified by Vidhya Gangadhara.

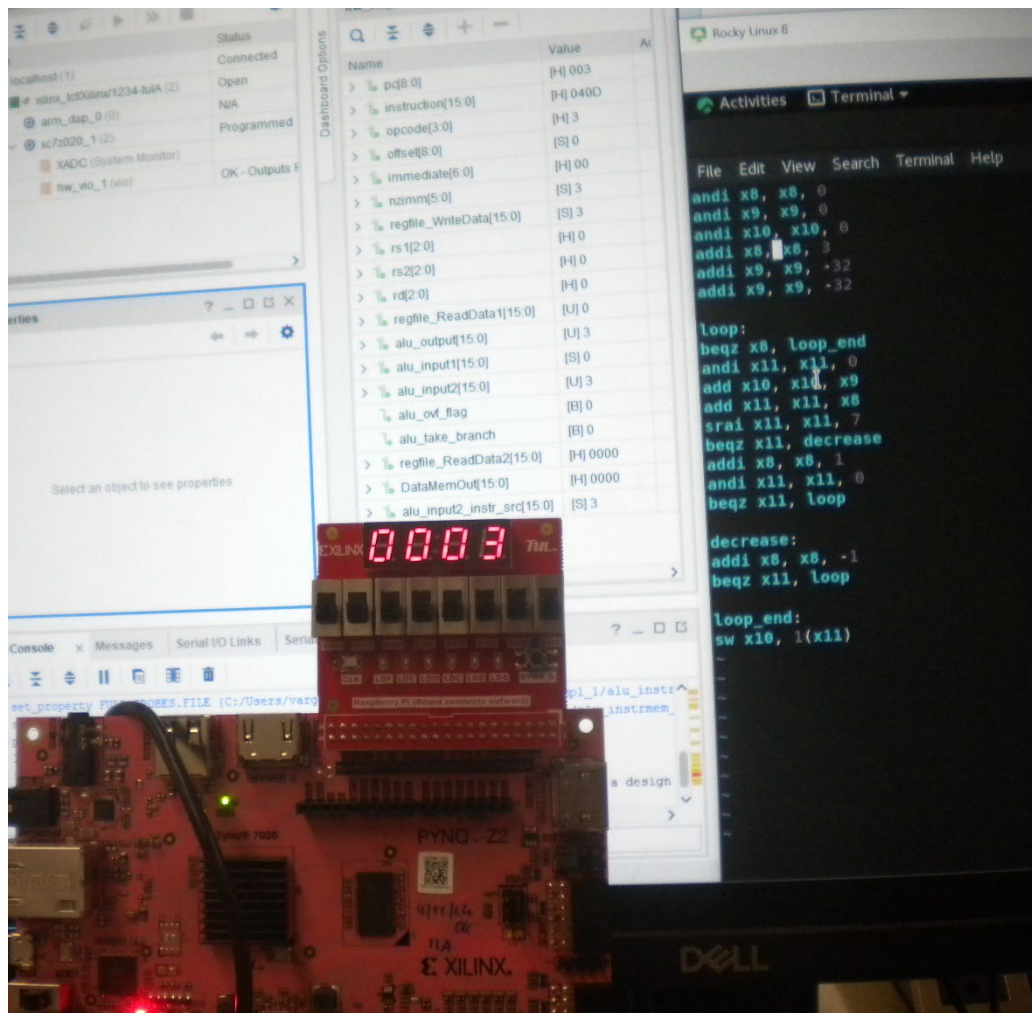


Figure 1: VIO and display output for third instruction, 0x040D.

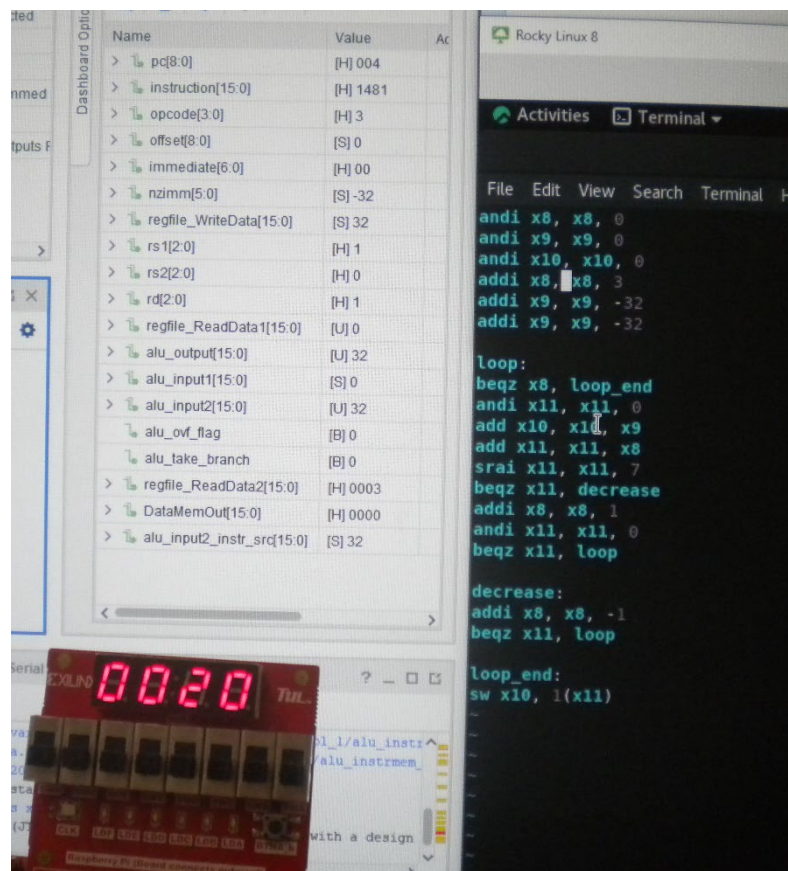


Figure 2: VIO and display output for fourth instruction, 0x1481.

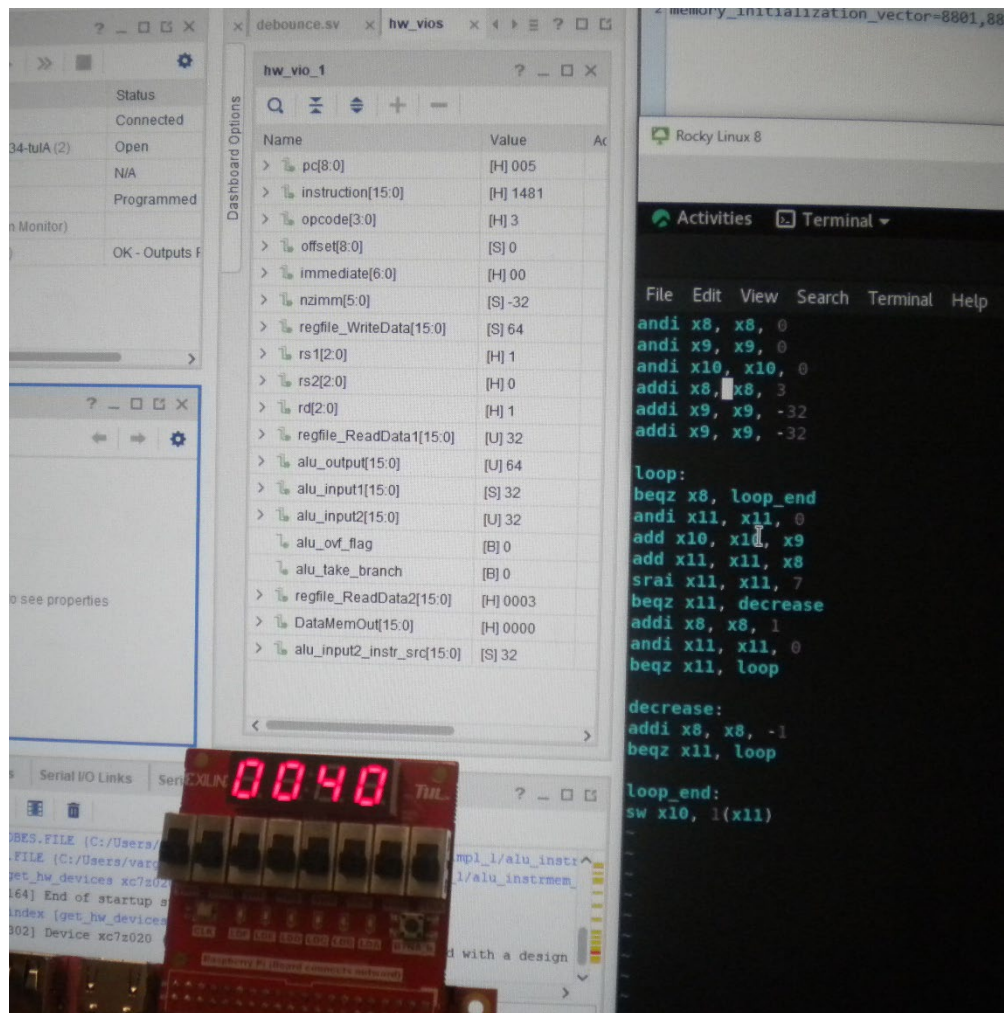


Figure 3: VIO and display output for fifth instruction, 0x1481.

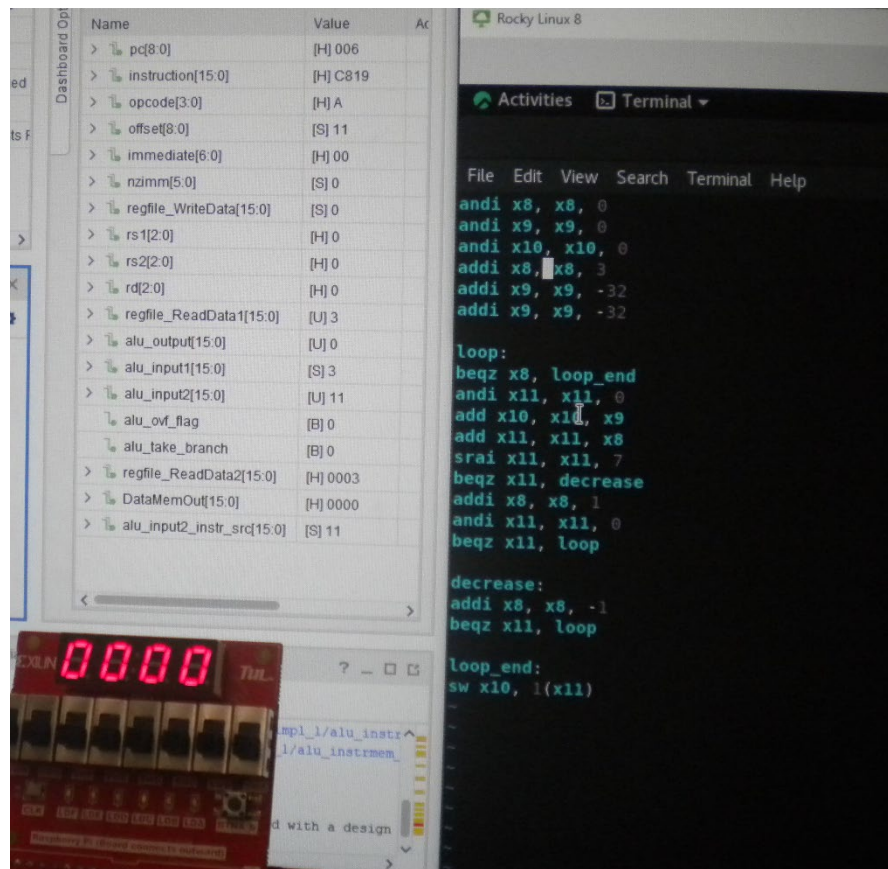


Figure 4: VIO and display output for sixth instruction, 0xC819.

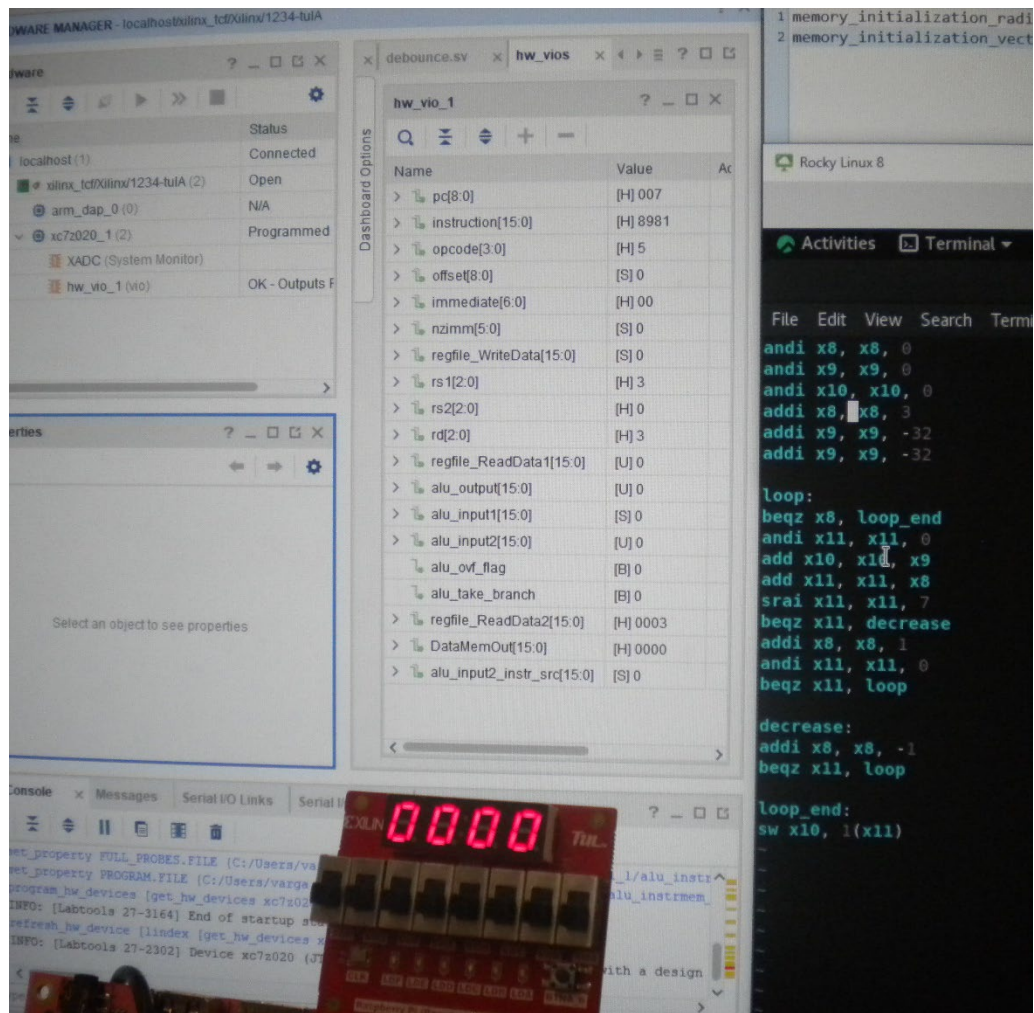


Figure 5: VIO and display output for seventh instruction, 0x8981.

5 Conclusion

In Lab 7, we wrote a RISC-V multiplier program, translated it to compressed machine code, and ran it on our computer. To accomplish this, we had to edit the program counter to add branching. From Lab 1 to Lab 7, we were able to create an independent single-cycle RISC-V processor.

Appendix A RISC-V Instruction Set

A.1 Initial Multiply Program

```
.data
num1: .word -32
num2: .word 5

.text
init:
lw x8, num1
lw x9, num2 #x9 is num of iterations

andi x10, x9, -256
bnez x10, negative #if num2 is negative, use the negative
beqz x10, positive #if num2AND-256 is positive, use positive cond

negative:
#negative second number
andi x10, x8, -256
#2's complement of x8 if the counter is negative
li x11, -1

xor x8, x8, x11
addi x8, x8, 1
li x11, 0
bnez x10, doubleneg
beqz x10, posneg

positive:
#positive second number
andi x10, x8, -256
beqz x10, pospos
bnez x10, negpos

pospos:
addi x9, x9, -1
add x11, x11, x8 #x8+x10
bnez x9, pospos
j end

posneg:
addi x9, x9, 1
#adds the 2's c
add x11, x11, x8
bnez x9, posneg
j end

negpos:
addi x9, x9, -1
```

```
add x11, x11, x8
bnez x9, negpos
j end
```

```
doubleneg:
#need to add the 2s-c
addi x9, x9, 1
add x11, x11, x8
bnez x9, doubleneg
j end
```

```
end:
li a7, 10
ecall
```

A.2 Functional Multiply Program

```
andi x8, x8, 0
andi x9, x9, 0
andi x10, x10, 0
addi x8, x8, 3
addi x9, x9, -32
addi x9, x9, -32
```

```
loop:
beqz x8, loop_end
andi x11, x11, 0
add x10, x10, x9
add x11, x11, x8
srai x11, x11, 7
beqz x11, decrease
addi x8, x8, 1
andi x11, x11, 0
beqz x11, loop
```

```
decrease:
addi x8, x8, -1
beqz x11, loop
```

```
loop_end:
sw x10, 1(x11)
```

A.3 Output.coe

```
memory_initialization_radix=16;
memory_initialization_vector=8801 8881 8901 040d 1481 1481 c819 8981 9526 95a2 859d c581 0405
8981 d9e5 147d d5f5;
```

Appendix B SystemVerilog Code

B.1 Program Counter

```
`timescale 1ns / 1ps
module program_counter(
    input logic clk, rst, take_branch,
    input logic[8:0] offset, output logic[8:0] pc
```



```
);  
always_ff@(posedge clk, posedge rst)  
if (rst) begin  
    pc <= 0;  
end  
else begin  
    if (take_branch) begin  
        pc <= pc + offset;  
  
    end  
    else pc <= pc + 1;  
end  
endmodule
```